Radial Basis Function Neural Network for Image Classification using CIFAR-10 Dataset Neural Networks - Deep Learning Course

Computer Science Department, Aristotle University of Thessaloniki

I. Introduction

THis project addresses the foundational problem of image classification using one of the most widely adopted benchmark datasets, CIFAR-10. Consisting of 60,000 tiny 32x32 color images across 10 distinct object classes, CIFAR-10 serves as an ideal testbed for evaluating the performance of simple, yet conceptually powerful, classification algorithms and deep learning models.

In contrast to the previous project which focused on prominent deep learning classifiers: the **Multilayer Perceptron (MLP)** and the **Convolutional Neural Network (CNN)** and SVMs, where we highlighted their fundamental architectural differences and their respective strengths and weaknesses when applied to image data, this section demonstrates the implementation of a **Radial Basis Function Neural Network** for the image classification of CIFAR-10.

Radial Basis Function

A Radial Basis Function is a real-valued function ϕ whose value depends solely on the distance between the input x and a fixed point c, called the center. Mathematically, it is expressed as:

$$\phi(x, c) = \phi(\|x - c\|)$$

An RBF network is a specialized feedforward architecture that transforms input data through a series of receptive fields. Unlike standard networks, it performs a nonlinear transformation in the hidden layer and a linear transformation in the output layer.

- **Input Layer:** This layer acts as a transparent layer that passes the n-dimensional input vector x to the hidden layer without any weight multiplication or bias addition. While a standard neuron in a neural network (like an MLP) performs a computation the moment it receives data, the input layer of an RBF network is mathematically unique because its weights are fixed at unity 1.0 and it has zero bias. In the context of CIFAR-10, the input is a $32 \times 32 \times 3$ image ($3,072$ pixels), so the input layer must have exactly $3,072$ nodes. Each node simply holds the pixel value.
- **Hidden RBF Layer:** This is the core of the system. Each of the M neurons in this layer represents a center c_i in the input space. The neuron calculates the distance between the input and its center, typically using the Euclidean Norm $\|x - c_i\|$. Each neuron in the hidden layer represents a prototype vector from the training set. The network computes the Euclidean distance between the input vector and each neuron's center. Because the input layer is transparent, the Hidden RBF Layer is the first place where learning occurs. The hidden layer receives the pure image and immediately operates to tell how far is a specific pixel pattern from a stored 'center' prototype. Every neuron in the hidden layer has a spread parameter σ and a center which are also referred to as prototype vectors. The spread parameter modulates the distance between the center of an RBF neuron and the input vector which in turn determines the neuron's output.
- **Output Nodes:** The output layer consists of 10 neurons (one for each CIFAR-10 class). Each neuron j calculates a weighted sum of all the RBF activations from the hidden layer:

$$y_j(x) = \sum_{i=1}^{M} w_{ji}\phi_i(x) + b_j$$

  where ϕ is the activation function from the RBF hidden neuron and w is the weight connecting hidden neuron i to output neuron j. This represents the "contribution" of that specific prototype to class j. To reduce the error between the network's predictions and the actual target values, the weights of these combinations are changed during training. Each output node calculates a score based on a weighted sum of the activation values from all RBF neurons. For classification the category with the highest score is chosen.

II. The CIFAR-10 Dataset and Vector Preparation

The CIFAR-10 dataset is a widely adopted benchmark in computer vision, consisting of $60,000$ color images, each of size 32×32 pixels. These images are equally distributed across 10 distinct object classes (e.g., airplane, automobile, bird, cat, dog), with $6,000$ images per class. The dataset is conventionally split into two subsets:
Training Set: $50,000$ images (N_{train}) used to learn the structure of the data.
Test Set: $10,000$ images (N_{test}) used for final evaluation.

A key challenge of CIFAR-10 is the low resolution (32×32), which forces NNs to rely on fine details, and the high intra-class variance (e.g., different angles and lighting for the same object), making generalization difficult for shallow models.
Vector Preparation for RBF Neural Network:

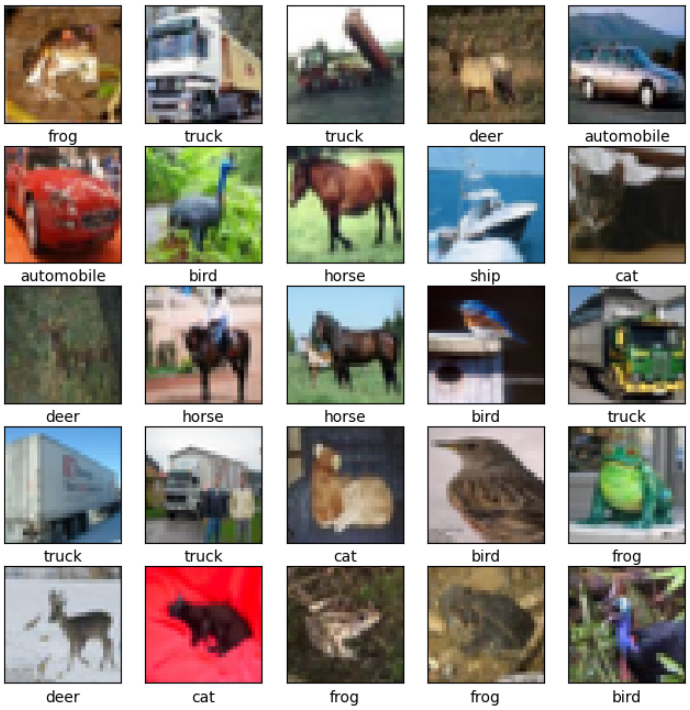

Fig. 1. Samples from every class of CIFAR-10.

Flattening: It is necessary for the Radial Basis Function. Each 32×32 color image has three color channels (Red, Green, Blue). To convert the image into a single feature vector $\mathbf{x}$, the dimensions are multiplied:

$$D = 32 \times 32 \times 3 = 3072 \text{ features}$$

Every image is thus represented by a 3072-dimensional vector. This process is necessary because an RBF's input layer expects a single vector for each sample.

Normalization: The raw pixel intensity values range from 0 to 255. To prevent features with inherently larger magnitude (such as those in the blue channel, if not properly scaled) from producing mathematically massive distances and thus "break" the formulas used to calculate similarity, this preprocessing step is required for the RBF, as it is distance-based algorithm. Furthermore, normalization prevents numerical overflows when calculating the exponential function, as extremely large distances could otherwise result in activation values that drop to zero too quickly, making the network's weights nearly impossible to train. If features are not standardized, those with larger numerical ranges will dominate the distance calculation, causing the network to ignore smaller but potentially more significant features. By transforming the data to have a mean of 0 and a standard deviation of 1, StandardScaler ensures that the feature space is isotropic. This allows the radial basis functions to spread uniformly across all dimensions, preventing the Gaussian centers from becoming biased and ensuring more stable numerical behavior during the training of the output weights.

Training Algorithm for RBF Neural Network

The training of a Radial Basis Function (RBF) neural network is an elegant hybrid approach that decouples the non-linear mapping of the hidden layer from the linear optimization of the output layer, starting with an unsupervised phase to determine the centers ($\mathbf{c}_i$) and widths (σ_i) of the basis functions. For the centers, K-Means clustering is typically employed to minimize the objective function $J = \sum_{j=1}^{K} \sum_{x \in S_j} \|\mathbf{x} - \mathbf{c}_j\|^2$, where the data is partitioned into K clusters S_j to represent the underlying distribution of the input space. Once the centers are established, the width or spread parameter σ_i for each neuron is calculated—often using the P-nearest neighbors heuristic $\sigma_i = \frac{1}{P}\sqrt{\sum_{j=1}^{P} \|\mathbf{c}_i - \mathbf{c}_j\|^2}$—to ensure that the Gaussian activation function, $\phi_i(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x}-\mathbf{c}_i\|^2}{2\sigma_i^2}\right)$, provides sufficient overlap and a smooth interpolation across the feature space.With the hidden layer parameters fixed, the network's mapping becomes a linear combination of these basis functions, expressed as $y(\mathbf{x}) = \sum_{i=1}^{K} w_i \phi_i(\mathbf{x}) + b$, where b is an optional bias term. This transforms the training of the output weights into a linear least-squares problem that can be solved globally and efficiently without the risk of getting stuck in local minima. If we define $\mathbf{\Phi}$ as the $N \times K$ design matrix where $\Phi_{ji} = \phi_i(\mathbf{x}_j)$, and $\mathbf{T}$ as the vector of target outputs, the optimal weight vector $\mathbf{W}$ is derived by minimizing the cost function $E = \frac{1}{2}\|\mathbf{T} - \mathbf{\Phi}\mathbf{W}\|^2$. This yields the closed-form solution via the Moore-Penrose pseudo-inverse, $\mathbf{W} = \mathbf{\Phi}^{\dagger}\mathbf{T} = (\mathbf{\Phi}^T\mathbf{\Phi})^{-1}\mathbf{\Phi}^T\mathbf{T}$, allowing for near-instantaneous training compared to the thousands of iterations required by backpropagation in standard Multi-Layer Perceptrons. In cases requiring extreme precision, a final supervised stage uses gradient descent to fine-tune the parameters by calculating the partial derivatives of the error with respect to the weights, centers, and widths:

$$\frac{\partial E}{\partial w_i} = -(t - y)\phi_i(\mathbf{x})$$

$$\frac{\partial E}{\partial \mathbf{c}_i} = w_i \phi_i(\mathbf{x}) \frac{(\mathbf{x} - \mathbf{c}_i)}{\sigma_i^2}(t - y)$$

$$\frac{\partial E}{\partial \sigma_i} = w_i \phi_i(\mathbf{x}) \frac{\|\mathbf{x} - \mathbf{c}_i\|^2}{\sigma_i^3}(t - y)$$

This comprehensive methodology ensures that the RBF network benefits from both the statistical representation of the data distribution and the mathematical rigor of linear algebra.

K-means Clustering for RBF Centers

When training a Radial Basis Function (RBF) neural network, K-Means clustering serves as the primary unsupervised mechanism for identifying the optimal locations of the hidden layer centers $\mathbf{c}_i$, ensuring that each neuron's activation function is "anchored" to a representative region of the input data space. K-Means is a centroid-based iterative algorithm that partitions a dataset into K distinct, non-overlapping clusters by minimizing the Within-Cluster Sum of Squares (WCSS), also known as the inertia objective function: $J = \sum_{j=1}^{K} \sum_{\mathbf{x} \in S_j} \|\mathbf{x} - \mathbf{c}_j\|^2$, where S_j represents the set of data points assigned to the jth cluster. The process begins by initializing K centers—where K corresponds exactly to the number of hidden neurons—either randomly or using a more strategic method like K-Means to ensure better initial spread. The algorithm then cycles through two main steps: the Assignment Step, where each input vector $\mathbf{x}_i$ is mapped to the nearest center based on the Euclidean distance $d(\mathbf{x}_i, \mathbf{c}_j) = \sqrt{\sum(\mathbf{x}_i - \mathbf{c}_j)^2}$, and the Update Step, where each center is recalculated as the arithmetic mean of all points assigned to it, expressed as $\mathbf{c}_j = \frac{1}{|S_j|} \sum_{\mathbf{x} \in S_j} \mathbf{x}$.
By using K-Means for center selection instead of random sampling, the RBF network effectively "discovers" the statistical density of the input space, placing more neurons where the data is dense and fewer where it is sparse. This creates a highly efficient "basis" for the subsequent supervised phase, as the hidden layer now represents a compressed version of the input distribution, allowing the final linear output layer to map these prototypes to the target values with maximum mathematical stability and minimal error.

Kernel Selection for RBF

In a Radial Basis Function Neural Network, the kernel (or basis function) acts as the hidden layer's activation function, transforming the Euclidean distance $r = \|\mathbf{x} - \mathbf{c}_i\|$ between an input vector and a neuron's center into a scalar value. These kernels determine the "receptive field" of each neuron—essentially defining how far an input's influence extends across the feature space. While the Gaussian kernel is the most popular due to its localized nature (it only responds to inputs near its center), other kernels offer different mathematical properties, such as global support or varying degrees of smoothness, which can be critical for tasks like surface interpolation, fluid dynamics modeling, or complex regression where local approximation alone isn't sufficient.

- **Gaussian Kernel:** The most widely utilized kernel due to its localized response and mathematical convenience. It is defined as:

$$\phi(r) = \exp\left(-\frac{r^2}{2\sigma^2}\right) \tag{1}$$

  where σ controls the width of the bell curve.
- **Multiquadric (MQ):** A global kernel that increases with distance, often used in topography and high-accuracy surface reconstruction:

$$\phi(r) = \sqrt{r^2 + \beta^2} \tag{2}$$

- **Inverse Multiquadric (IMQ):** A localized version of the MQ kernel that possesses "heavy tails," allowing for a broader influence than the Gaussian:

$$\phi(r) = \frac{1}{\sqrt{r^2 + \beta^2}} \tag{3}$$

- **Thin Plate Spline:** Highly effective for coordinate transformations and image warping as it minimizes the "bending energy" of a surface:

$$\phi(r) = r^2 \ln(r) \tag{4}$$

- **Inverse Quadratic:** A computationally efficient alternative to the Gaussian that decays at a power-law rate rather than exponentially:

$$\phi(r) = \frac{1}{1 + (\epsilon r)^2} \tag{5}$$

III. Radial Basis Function with Gaussian Kernel

For our final model, we chose the Gaussian Kernel because it yelded the best performance among all other kernels. An RBF Neural Network utilizing a Gaussian kernel functions as a high-dimensional mapping system that transforms an input vector $\mathbf{x} \in \mathbb{R}^d$ into a linearized feature space through a hidden layer of K radially symmetric basis functions. Mathematically, the response of each hidden neuron is governed by the Gaussian function $\phi_i(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x}-\mathbf{c}_i\|^2}{2\sigma_i^2}\right)$, where the numerator represents the squared Euclidean distance (or L_2 norm) between the stochastic input and a learned prototypical centroid $\mathbf{c}_i$, while the denominator σ_i^2 serves as the bandwidth parameter or variance that dictates the spatial extent of the neuron's receptive field. This architecture is predicated on Cover's Theorem on the Separability of Patterns, which posits that a complex pattern-classification problem cast non-linearly into a high-dimensional space is more likely to be linearly separable than in a low-dimensional space. The hidden layer acts as a non-linear "coordinate transformer," where the activation $\phi_i(\mathbf{x})$ reaches its maximum value of unity when $\mathbf{x} = \mathbf{c}_i$ and decays exponentially as the input moves toward the peripheral regions of the feature space, effectively creating a localized approximation of the underlying function. The global output is then computed as a linear regression of these activations, defined by the summation $y(\mathbf{x}) = \sum_{i=1}^{K} w_i \phi_i(\mathbf{x}) + b$, where w_i are the synaptic weights of the output layer. Training this system involves an unsupervised optimization phase—typically via the Expectation-Maximization (EM) algorithm or K-Means clustering—to establish the topological distribution of $\mathbf{c}_i$, followed by a supervised phase to solve the Normal Equations $\mathbf{W} = (\mathbf{\Phi}^T\mathbf{\Phi})^{-1}\mathbf{\Phi}^T\mathbf{T}$. This hybrid methodology allows the RBF network to function as a Universal Approximator, capable of modeling any continuous function $f(\mathbf{x})$ by superposing localized Gaussian "bumps" that collectively interpolate the data manifold with high mathematical stability and minimal risk of convergence into local minima, unlike iterative backpropagation-based architectures.

Model Architecture:

- **Preprocessing**: The first step applies Z-score normalization, mathematically defined as $1z = \frac{x-\mu}{\sigma}$.2 The pipeline

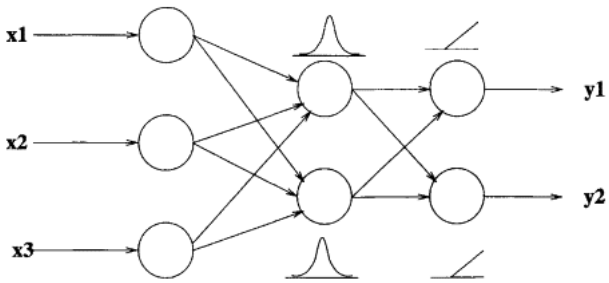

Fig. 2. Three Stage Architecture of RBF with Gaussian Kernel.

first applies standard scaling to normalize pixel values to a mean of 0 and variance of 1. In the context of raw images (where pixels range from 0 to 255), this transformation is critical to prevent feature dominance and improve the optimization procedure as we discussed earlier. Thus the RBF treats all features equally and converges quickly.

- **Dimensionality Reduction**: The second step addresses the **Curse of Dimensionality**. A raw CIFAR-10 image has 3,072 dimensions (features). In such high-dimensional spaces, data points become incredibly sparse, making it statistically difficult to find a robust separating hyperplane without overfitting. The parameter n_components=0.90 instructs the algorithm to calculate the eigenvalues of the covariance matrix of the features and keep only the top k eigenvectors, which are the principal components required to preserve 90% of the original information (variance). The algorithm discards noise and redundant pixels, keeping only the principal features that account for 95% of the image variance. This shrinks the input vector size and speeds up training. For CIFAR-10 this reduces the feature count from 3,072 to fewer than 200 components.
- **Number of Prototypes**: The parameter n_hidden represents the number of neurons in the hidden layer, which corresponds to the number of centers ($\mathbf{c}_i$) identified during the K-Means clustering phase. Each of these 2,500 neurons acts as a "prototype" or a landmark in the high-dimensional PCA-reduced space. By choosing 2,500, you are asking the K-Means algorithm to partition the CIFAR-10 dataset into 2,500 distinct clusters. A higher number of hidden neurons increases the network's ability to capture complex, non-linear boundaries. In mathematical terms, your design matrix $\mathbf{\Phi}$ will have 2,500 columns. This allows the linear output layer to form a more complex weighted sum:

$$y(\mathbf{x}) = \sum_{i=1}^{2500} w_i \phi_i(\mathbf{x})$$

- **Spread Parameter**: The parameter sigma (σ) is the width of the Gaussian kernel. It dictates the receptive field of each hidden neuron—essentially defining how far an input $\mathbf{x}$ can be from a center $\mathbf{c}_i$ before the neuron stops firing. The Gaussian activation is calculated as $\exp(-\frac{d^2}{2\sigma^2})$. With $\sigma = 100$, the denominator $2\sigma^2$ becomes $20,000$. This suggests that your input features (after PCA and scaling) have relatively large Euclidean distances. The Gaussian "bumps" are wide and overlap significantly. This helps the network generalize by averaging the influence of many nearby prototypes.

Training Analysis and Evaluation:

The model's performance was rigorously evaluated using the Loss function is defined as the Sum of Squared Errors (SSE) or Mean Squared Error (MSE) between the target matrix Y and the predicted output $W\Phi$. Mathematically, the objective is to minimize the L_2 norm: $L = \|\Phi W - Y\|_2^2$, where Φ represents the activation matrix of the hidden layer. Unlike traditional deep learning models that minimize loss iteratively via Stochastic Gradient Descent, the Moore-Penrose Pseudo-inverse ($\Phi^\dagger$) provides the unique, optimal analytical solution that achieves the global minimum of this quadratic error surface in a single computational step. By calculating $W = \Phi^\dagger Y = (\Phi^T\Phi)^{-1}\Phi^T Y$, the algorithm finds the weights that project the target labels onto the column space of the hidden layer activations.

The accuracy of roughly **53%** for the RBF Neural Network with Gaussian kernel on the CIFAR-10 dataset highlights the fundamental mismatch between the model's simplicity and the data's complexity. First, while it is better than the random baseline loss for a 10-class problem (which is $\approx 10\%$), it remains far from an acceptable level for modern image classification, indicating the model is only marginally learning the task. CIFAR-10 images are 3,072-dimensional vectors (flattened $32 \times 32 \times 3$ pixel grids) where classes like "cat" or "airplane" are defined by complex, non-linear manifolds—meaning their patterns curve and twist in high-dimensional space. The 53% threshold indicates that while the 2,500 K-Means centers successfully capture the "global" statistical regularities of the classes—such as general color distributions and coarse shapes—the model struggles with the high intra-class variance inherent in CIFAR-10. A σ of 100 likely results in significant "activation overlap" where the Gaussian basis functions become too broad, causing the hidden layer to lose its discriminative "sharpness" and resulting in a sub-optimal linear separation in the output stage.

```
Training Accuracy: 59.24
Testing Accuracy: 52.59

Classification Report:
              precision    recall  f1-score   support

           0       0.60      0.61      0.60      1000
           1       0.62      0.67      0.64      1000
           2       0.42      0.34      0.38      1000
           3       0.39      0.31      0.34      1000
           4       0.46      0.40      0.43      1000
           5       0.48      0.41      0.45      1000
           6       0.49      0.66      0.56      1000
           7       0.57      0.60      0.59      1000
           8       0.60      0.70      0.64      1000
           9       0.56      0.57      0.57      1000

    accuracy                           0.53     10000
   macro avg       0.52      0.53      0.52     10000
weighted avg       0.52      0.53      0.52     10000
```

Fig. 3. Train, Test and Per-Class Accuracy

Heat Map Analysis:

The Heatmap reveals the physical manifestation of this calculation: the extreme weight values—ranging from -200 to +300—indicate a state of numerical instability or an ill-conditioned activation matrix Φ. Because a σ of 100 creates broad, overlapping Gaussian basis functions, the pseudo-inverse is forced to assign massive coefficients to tiny differences in neuron activations to distinguish between classes, a phenomenon clearly visible in "expert" neurons like Neuron 14. This aggressive weighting allows the model to capture global statistical regularities (like color and coarse shape) but causes it to "over-fit" to training noise rather than learning fine-grained, generalizable features. Consequently, the 53% accuracy marks the linear bottleneck of the system, where the unsupervised K-Means centers and wide RBFs can no longer provide a sufficiently sharp non-linear mapping to resolve the complex intra-class variances of CIFAR-10. Scientifically, the color intensities represent the weight coefficients w_{ij} that determine the influence of each hidden neuron on the final decision score for a specific category. The observed weight magnitudes—spanning from -200 to +300—are a direct consequence of the Moore-Penrose Pseudo-inverse solving a system where the activation matrix Φ is ill-conditioned due to the high overlap caused by $\sigma = 100$. Because the Gaussian basis functions are too broad, the neurons produce highly similar activation patterns; the mathematical solver compensates for this lack of "orthogonality" by assigning extreme, "aggressive" weights to the minor variations between neurons to force a separation between classes. This results in "fragile" neurons, such as Neuron 14, which exerts a disproportionately high influence on Class 2 while heavily inhibiting Class 0. While this configuration minimizes the Training Mean Squared Error (MSE), the high contrast in the heatmap confirms that the model is relying on numerical artifacts rather than robust, generalizable visual features, which explains why the accuracy plateaus at 53%.

Confusion Matrix Analysis:
The matrix exposes model's real-world failures, which explains why the 53% accuracy feels like a ceiling. Scientifically, this matrix maps the ground truth against the model's predictions, revealing that the "linear bottleneck" is most severe among visually similar classes. For instance, the high error rates between cats and dogs (164 misclassifications) or trucks and automobiles (180 misclassifications) occur because the 2,500 unsupervised K-Means centers have not extracted features sharp enough to distinguish these categories in a high-dimensional space. The model effectively captures "coarse" features (ships and airplanes), but the fine-grained nuances are lost in the overlapping Gaussian bells. It shows that the model often picks the "neighboring" class (e.g., predicting a truck as an automobile). This happens because the broad $\sigma = 100$ Gaussians cause the same "expert" neurons in the heatmap to fire for both vehicles, making their output scores nearly identical.

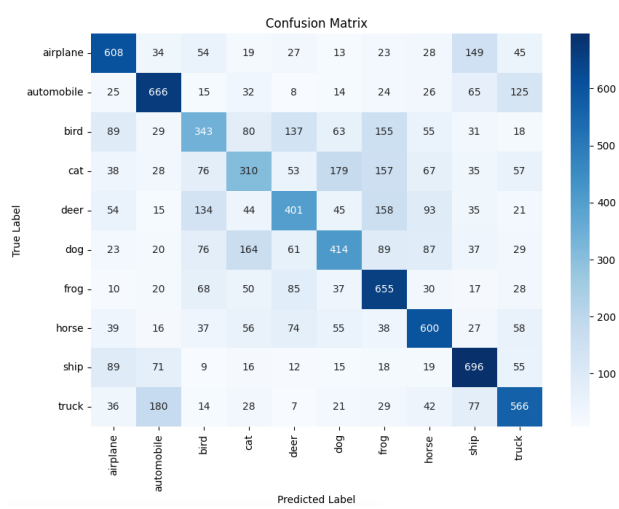

Fig. 4. Confusion Matrix for RBF with Gaussian Kernel.